



الهيئة الاتحادية للهوية
والجنسية والجمارك وأمن المنافذ
FEDERAL AUTHORITY FOR IDENTITY,
CITIZENSHIP, CUSTOMS & PORT SECURITY

ID Card Toolkit Developer Guide

JavaScript

Version 1.22

March 2026

CONFIDENTIAL

Table of Contents

Revision History	4
Abbreviations	5
1 Introduction	6
1.1 Document Purpose	6
2 Development Prerequisites & Environment Setup	7
2.1 Prerequisites	7
2.2 Environment Setup	7
3 Developing Your First Program	8
3.1 Step 1: Initialize the Toolkit	8
3.2 Step 2: List Connected Readers	9
3.3 Step 3: Read Public Data	9
3.4 Step 4: Disconnect	10
4 Toolkit API Reference	11
4.1 Class: Toolkit	11
4.1.1 Constructor: Toolkit	11
4.1.2 Method: listReaders	11
4.1.3 Method: getReaderWithEmiratesId	11
4.1.4 Method: prepareRequest	12
4.1.5 Method: registerDevice	12
4.1.6 Method: getToolkitVersion	12
4.1.7 Method: cleanup	13
4.1.8 Method: getDeviceId	13
4.1.9 Method: parseMRZ	13
4.1.10 Method: getConfigFilesExpiryDates	13
4.1.11 Method: getLicenseExpiryDate	13
4.2 Class: CardReader	14
4.2.1 Method: connect	14
4.2.2 Method: disconnect	14
4.2.3 Method: getCardVersion	14
4.2.4 Method: getReaderName	14
4.2.5 Method: getReaderSerialNumber	15
4.2.6 Method: getCardSerialNumber	15
4.2.7 Method: getInterfaceType	15
4.2.8 Method: readPublicData	15
4.2.9 Method: readPublicDataEF	16
4.2.10 Method: checkCardStatus	16
4.2.11 Method: prepareRequest (CardReader)	17
4.2.12 Method: getPkiCertificates	17
4.2.13 Method: authenticatePki	17
4.2.14 Method: signData	18
4.2.15 Method: signChallenge	18
4.2.16 Method: verifySignature	18
4.2.17 Method: getFingerData	19
4.2.18 Method: authenticateBiometricOnServer	19
4.2.19 Method: authenticateBiometricandCardOnServer	19

4.2.20 Method: resetPin	20
4.2.21 Method: resetPINWithoutAuthenticateBiometric	20
4.2.22 Method: unblockPin	20
4.2.23 Method: readFamilyBookData	21
4.2.24 Method: setNfcAuthenticationParameters (Card Attributes)	21
4.2.25 Method: setNfcAuthenticationParameters (MRZ)	21
4.3 ParseEFData	22
4.3.1 Method: parseEFData	22
5 Data Model Classes	23
5.1 Class: ToolkitException	23
5.2 Class: ToolkitExceptionWithResponse	23
5.3 Class: ToolkitResponse	23
5.4 Class: RegisterDeviceResponse	24
5.5 Class: CardCertificates	24
5.6 Class: FingerData	24
5.7 Class: SignatureResponse	24
5.8 Class: CardPublicData	24
5.9 Class: CardFamilyBookData	25
6 Advanced Digital Signature API Reference	26
6.1 Class: CardReader (Digital Signature Methods)	26
6.1.1 Method: padesSign	26
6.1.2 Method: padesVerify	26
6.1.3 Method: xadesSign	26
6.1.4 Method: xadesVerify	27
6.1.5 Method: cadesSign	27
6.1.6 Method: cadesVerify	28

Revision History

Version	Date	Description of Changes
1.0	5th Feb, 2017	Beta Release 1
1.1	18th Mar, 2017	Beta Release 2
1.2	17th Apr, 2017	Beta Release 3
1.3	10th Dec, 2017	Release Version 1.0.0
1.5	02nd May, 2018	Release Version 1.0.5
1.6	03rd Jul, 2018	Release Version 1.0.7
1.7	20th Mar, 2019	Release Version 1.1.0
1.8	03rd Apr, 2019	Release Version 1.2.0
1.9	13th May, 2019	Release Version 1.3.0
1.10	07th Jun, 2019	Release Version 1.4.0
1.11	17th Jun, 2019	Release Version 1.4.1
1.12	19th Dec, 2019	Release Version 1.4.4
1.13	04th Mar, 2020	Release Version 1.4.5
1.14	01st Apr, 2020	Release Version 1.4.5-r2
1.15	05th Apr, 2020	Release Version 1.4.6
1.16	24th May, 2021	Release Version 1.4.9
1.17	04th Oct, 2021	Release Version 2.0.2
1.18	12th Apr, 2022	Release Version 2.0.3
1.19	12th Jun, 2022	Release Version 2.0.5
1.20	20th Dec, 2023	Release Version 3.0.0
1.21	20th May, 2025	Release Version 3.0.3-r1
1.22	Mar 2026	Toolkit v3.0.5. Improved code samples.

Abbreviations

Abbreviation	Description
CA	Certificate Authority
CAdES	CMS Advanced Electronic Signatures
CMS	Cryptographic Message Syntax
EF	Elementary File
ICP	Federal Authority for Identity, Citizenship, Customs & Port Security
IDE	Integrated Development Environment
JNLP	Java Network Launch Protocol
MRZ	Machine Readable Zone
NFC	Near Field Communication
PAdES	PDF Advanced Electronic Signatures
PDF	Portable Document Format
SP	Service Provider
VG	Validation Gateway
XAdES	XML Advanced Electronic Signatures
XML	Extensible Markup Language

1. Introduction

The Federal Authority for Identity, Citizenship, Customs & Port Security (ICP) developed the ID Card Toolkit to address the requirements of service providers (SP) to integrate, into their business applications: Identification, Authentication, Digital Signature and Non-repudiation services, around the capabilities of the Emirates ID Card. The Toolkit is comprised of a number of Software Development Kits (SDK) supporting different programming languages and platforms.

1.1. Document Purpose

This document is a developer's guide to support the development of JavaScript applications that integrate the ICP ID Card Toolkit SDK.

2. Development Prerequisites & Environment Setup

2.1. Prerequisites

- Toolkit SDK Web distribution
- Toolkit Service (running on client machine)
- JavaScript IDE (e.g., Eclipse IDE, VS Code)
- Latest web browser (Firefox 50+, Chrome, Edge)
- Apache Tomcat 8+ server (or equivalent)

2.2. Environment Setup

1. Download the latest Toolkit JavaScript SDK from the ICP web site.
2. Add `eidatoolkit.js` to your project.
3. Include `eidatoolkit.js` in your HTML pages.

3. Developing Your First Program

The following example shows the sequence of API calls required to develop an application using the Toolkit JavaScript SDK. The example illustrates how to use the Read Public Data service. To use other services, follow a similar approach.

Refer to the Toolkit API Reference (Section 4) for details of all available services. A sample program is provided with the SDK.

3.1. Step 1: Initialize the Toolkit

Toolkit initialization is performed by creating a new `Toolkit` object with three callback functions and a configuration options object.

```
// Configuration options
var configOptions = {
  "jnlp_address": "http://xx.xx.xx.xx:xx/xx",
  "debugEnabled": false,
  "toolkitConfig": 'vg_url = "http://192.168.1.34:90/VGPreProd"\n'
    + 'vg_connection_timeout = 60\n'
    + 'configuration = "dev"\n'
    + 'config_location_online = false\n'
    + 'log_level = "INFO"'
};

// Callback: connection established
var onOpenCB = function (response, error) {
  if (error === null) {
    ToolkitOB.listReaders(listReaderCB);
  } else {
    ToolkitOB = null;
  }
};

// Callback: connection closed
var onCloseCB = function (event) {
  ToolkitOB = null;
  readerClass = null;
};

// Callback: communication error
var onErrorCB = function (err) {
  readerClass = null;
  if (null !== err) {
    alert('Error: ' + err);
  }
};

// Create Toolkit instance
ToolkitOB = new Toolkit(onOpenCB, onCloseCB, onErrorCB, configOptions);
```


3.2. Step 2: List Connected Readers

Retrieve a list of connected smartcard readers using `listReaders`. The callback receives an array of `CardReader` objects.

```
ToolkitOB.listReaders(listReaderCB);

var listReaderCB = function (response, error) {
  var readerList = response;
  if (0 < readerList.length) {
    readerClass = readerList[0];
  } else {
    return 'No readers found';
  }
  console.log('readerName = ' + readerClass.getReaderName());
  readerClass.connect(connectReaderCB);
};

var connectReaderCB = function (response, error) {
  if (null !== response) {
    console.log('Connected to the reader..');
  }
  if (null !== error) {
    return error.code + ' : ' + error.message;
  }
};
```

Note: To directly access a card reader with an Emirates ID card inserted, use `getReaderWithEmiratesId`:

```
ToolkitOB.getReaderWithEmiratesId(function (response, error) {
  if (response !== null) {
    readerClass = response;
    readerClass.connect(connectReaderCB);
  } else {
    console.log(error);
  }
});
```

3.3. Step 3: Read Public Data

Generate a Request ID (40-byte random, Base64-encoded) and call `readPublicData`. The callback receives a `CardPublicData` object on success.

```
readerClass.readPublicData(
  requestId,
  readNonModifiableData,
  readModifiableData,
  readPhotography,
  readSignatureImage,
  readAddress,
  function (response, error) {
```

```
        if (error === null) {  
            var cardPublicData = response;  
            console.log('Id Number = ' + cardPublicData.idNumber);  
        } else {  
            console.log(error);  
        }  
    }  
};
```

3.4. Step 4: Disconnect

Disconnect from the reader when done.

```
readerClass.disconnect(function (response, error) {  
    if (error !== null) {  
        alert(error.message);  
    }  
});
```

4. Toolkit API Reference

4.1. Class: Toolkit

This class provides methods required to initialize the Toolkit and manage connected smartcard readers. All asynchronous methods accept a callback function (`appCallback`) as the last parameter. The callback signature is: `function(response, error)` where `response` contains the result on success (`null` on error) and `error` is a `ToolkitException` object on failure (`null` on success).

4.1.1. Constructor: Toolkit

Initializes the Toolkit context and establishes a WebSocket connection to the Toolkit Service.

Syntax

```
ToolkitOB = new Toolkit(onOpenCB, onCloseCB, onErrorCB, options);
```

Parameter	Description
<code>onOpenCB</code>	Callback executed when the connection to the Toolkit Service is established. Signature: <code>function(response, error)</code> . On success, <code>response</code> contains 'success'.
<code>onCloseCB</code>	Callback executed when the connection is closed. Signature: <code>function(event)</code> . The <code>event</code> parameter contains the close reason.
<code>onErrorCB</code>	Callback executed on communication errors. Signature: <code>function(error)</code> . The <code>error</code> parameter contains the error message.
<code>options</code>	Configuration object with the following keys: • <code>jnlp_address</code>: URL to download the Toolkit Service • <code>debugEnabled</code>: <code>true/false</code> for debug mode • <code>toolkitConfig</code>: Configuration parameters string. Refer to the Programmer's Reference for details. Pass <code>null</code> to use the default <code>config_ap</code> file.

Return: Toolkit object.

4.1.2. Method: listReaders

Lists all compatible smartcard readers connected to the system.

Syntax

```
listReaders(appCallback);
```

Callback response: `CardReader[]` – array of connected smartcard readers. Empty array if none found.

4.1.3. Method: getReaderWithEmiratesId

Checks all smartcard readers and returns the first reader with an Emirates ID Card inserted.

Syntax

```
getReaderWithEmiratesId(appCallback);
```

Callback response: CardReader – the reader with an Emirates ID Card inserted.

4.1.4. Method: prepareRequest

Establishes a secure context for execution of a Toolkit service. The returned Request Handle is used to encode sensitive information (User ID, Password, PIN).

Syntax

```
prepareRequest(requestId, appCallback);
```

Parameter	Description
requestId	Application-generated request identifier. Refer to the Programmer's Reference for guidelines.

Callback response: String – 8-byte request handle in Base64 format.

4.1.5. Method: registerDevice

Registers a device with the Validation Gateway (VG). `prepareRequest` must be called first.

Syntax

```
registerDevice(encodedUserId, encodedPassword, deviceReferenceId, appCallback);
```

Parameter	Description
encodedUserId	SP User ID, encrypted and Base64-encoded per the Programmer's Reference.
encodedPassword	Password, encrypted and Base64-encoded per the Programmer's Reference.
deviceReferenceId	Unique device ID generated by the SP.

Callback response: RegisterDeviceResponse – contains deviceRegistrationID.

4.1.6. Method: getToolkitVersion

Returns the Toolkit version.

Syntax

```
getToolkitVersion(appCallback);
```

Callback response: String – version in major.minor.patch format.

4.1.7. Method: cleanup

Releases all Toolkit resources and closes the service context.

Syntax

```
cleanup();
```

Parameters: None. This method is synchronous.

4.1.8. Method: getDeviceId

Retrieves the unique device identifier, required for device deregistration.

Syntax

```
getDeviceId(appCallback);
```

Callback response: String – the device identifier.

4.1.9. Method: parseMRZ

Parses a Machine Readable Zone (MRZ) string and retrieves the corresponding attributes.

Syntax

```
parseMRZ(mrzString, appCallback);
```

Parameter	Description
mrzString	The MRZ string to parse.

Callback response: MRZData – parsed MRZ attributes.

4.1.10. Method: getConfigFilesExpiryDates

Returns the expiry dates of certificates corresponding to the Toolkit SDK configuration files.

Syntax

```
getConfigFilesExpiryDates(appCallback);
```

Callback response: String – certificate expiry dates for configuration files.

4.1.11. Method: getLicenseExpiryDate

Returns the expiry date of the Toolkit SDK license.

Syntax

```
getLicenseExpiryDate(appCallback);
```

Callback response: String – the license expiry date.

4.2. Class: CardReader

This class provides all methods required to perform operations with the Emirates ID Card after a successful connect operation.

4.2.1. Method: connect

Establishes a connection to the ID card in the reader. Must be called each time a card is presented or the reader is reconnected. Reusing the connection for multiple services improves performance.

Syntax

```
connect(appCallback);
```

Callback response: 'success' on successful connection.

4.2.2. Method: disconnect

Disconnects from the connected reader. The card can be safely removed after disconnect.

Syntax

```
disconnect(appCallback);
```

Callback response: 'success' on successful disconnect.

4.2.3. Method: getCardVersion

Retrieves the version number of the connected Emirates ID Card.

Syntax

```
getCardVersion(appCallback);
```

Callback response: Object containing card_version attribute.

Value	Card Version
2	V2
3	V3
4	V4
5	V5

V5 is the latest generation card with enhanced security. Full read, PKI, and biometric operations are supported.

4.2.4. Method: getReaderName

Retrieves the name of the connected smartcard reader. This is a synchronous method.

Syntax

```
String getReaderName();
```

Return: String – the reader name.

4.2.5. Method: getReaderSerialNumber

Retrieves the serial number of the connected smartcard reader. This is a synchronous method.

Syntax

```
String getReaderSerialNumber();
```

Return: String – the reader serial number, or null if not available.

4.2.6. Method: getCardSerialNumber

Retrieves the serial number of the connected Emirates ID Card.

Syntax

```
getCardSerialNumber(appCallBack);
```

Callback response: String – the card serial number.

4.2.7. Method: getInterfaceType

Returns the card communication interface type.

Syntax

```
getInterfaceType(appCallBack);
```

Callback response: Interface type number.

Value	Interface
1	Contact
2	NFC

4.2.8. Method: readPublicData

Retrieves data stored on the public areas of the Emirates ID Card. Flags control which data categories are returned.

Syntax

```
readPublicData(requestId, readNonModifiableData, readModifiableData,  
readPhotography, readSignatureImage, readAddress, appCallBack);
```

Parameter	Description
requestId	Application-generated request identifier.
readNonModifiableData	true to read non-modifiable data.
readModifiableData	true to read modifiable data.
readPhotography	true to read the card holder photograph.
readSignatureImage	true to read the handwritten signature image.
readAddress	true to read home and work address data.

Callback response: CardPublicData – contains the requested public data. The `xmlString` property returns the digitally signed XML response.

4.2.9. Method: readPublicDataEF

Retrieves Elementary File data from the public areas of the ID card.

Syntax

```
readPublicDataEF(publicDataEFTYPE, appCallback);
```

Parameter	Description
publicDataEFTYPE	Elementary File identifier (see table below).

Value	Constant	EF Data
1	IDN_CN	Identification and Card Number
2	ROOT_CERTIFICATE	Root Certificate
3	NON_MODIFIABLE_DATA	Non-Modifiable Public Data
4	MODIFIABLE_DATA	Modifiable Public Data
5	PHOTOGRAPHY	Photography
6	SIGNATURE_IMAGE	Handwritten Signature Image
7	HOME_ADDRESS	Home Address
8	WORK_ADDRESS	Work Address

Callback response: ToolkitResponse containing `ef_raw_data` for the requested EF data.

4.2.10. Method: checkCardStatus

Validates the Emirates ID Card with the VG and returns the card status.

Syntax


```
checkCardStatus(requestId, appCallBack);
```

Parameter	Description
requestId	Application-generated request identifier.

Callback response: ToolkitResponse – contains the card status. The xmlString property returns the XML response for signature validation.

4.2.11. Method: prepareRequest (CardReader)

Establishes a secure context for execution of a Toolkit service on the connected card.

Syntax

```
prepareRequest(requestId, appCallBack);
```

Parameter	Description
requestId	Application-generated request identifier.

Callback response: String – 8-byte request handle in Base64 format.

4.2.12. Method: getPkiCertificates

Reads the authentication and signing certificates from the Emirates ID Card. prepareRequest must be called first.

Syntax

```
getPkiCertificates(encodedPin, appCallBack);
```

Parameter	Description
encodedPin	ID Card PKI PIN, encrypted and Base64-encoded per the Programmer's Reference.

Callback response: CardCertificates – contains authenticationCertificate and signingCertificate as Base64-encoded strings.

4.2.13. Method: authenticatePki

Authenticates the ID Card PIN and verifies the revocation status of the authentication certificate. prepareRequest must be called first.

Syntax

```
authenticatePki(encodedPin, appCallBack);
```

Parameter	Description
encodedPin	ID Card PKI PIN, encrypted and Base64-encoded per the Programmer's Reference.

Callback response: ToolkitResponse – contains the PKI authentication status.

4.2.14. Method: signData

Digitally signs plain data or a hash using the signature certificate of the ID card. `prepareRequest` must be called first.

Syntax

```
signData(input, isInputHash, encodedPin, appCallback);
```

Parameter	Description
input	Data or hash to digitally sign.
isInputHash	true if hash data is provided; false if plain data.
encodedPin	ID Card PKI PIN, encrypted and Base64-encoded per the Programmer's Reference.

Callback response: SignatureResponse – contains signature (Base64-encoded).

4.2.15. Method: signChallenge

Digitally signs challenge data using the authentication certificate of the ID card. `prepareRequest` must be called first.

Syntax

```
signChallenge(input, isInputHash, encodedPin, appCallback);
```

Parameter	Description
input	Challenge bytes to digitally sign.
isInputHash	true if hash data is provided; false if plain challenge data.
encodedPin	ID Card PKI PIN, encrypted and Base64-encoded per the Programmer's Reference.

Callback response: SignatureResponse – contains signature (Base64-encoded).

4.2.16. Method: verifySignature

Verifies a digital signature with the corresponding certificate.

Syntax

```
verifySignature(input, isInputHash, signature, certificate, appCallBack);
```

Parameter	Description
input	Data or hash corresponding to the digital signature.
isInputHash	true if hash data is provided; false if plain data.
signature	The digital signature to verify.
certificate	Certificate data for signature verification.

Callback response: 'success' on successful verification.

4.2.17. Method: `getFingerData`

Returns finger index values and biometric reference identifiers of the fingerprints stored on the card.

Syntax

```
getFingerData(appCallBack);
```

Callback response: `FingerData[]` – array of finger data objects.

4.2.18. Method: `authenticateBiometricOnServer`

Performs biometric authentication with the VG service against a captured finger image.

Syntax

```
authenticateBiometricOnServer(requestId, fingerIndex, sensorTimeout, appCallBack);
```

Parameter	Description
requestId	Application-generated request identifier.
fingerIndex	Finger index from a previous <code>getFingerData</code> call.
sensorTimeout	Timeout in seconds for the fingerprint sensor.

Callback response: `ToolkitResponse` – contains the biometric authentication status.

4.2.19. Method: `authenticateBiometricandCardOnServer`

Validates the Emirates ID Card and performs biometric authentication with the VG service.

Syntax

```
authenticateBiometricandCardOnServer(requestId, fingerIndex, sensorTimeout, appCallBack);
```

Parameter	Description
requestId	Application-generated request identifier.
fingerIndex	Finger index from a previous <code>getFingerData</code> call.
sensorTimeout	Timeout in seconds for the fingerprint sensor.

Callback response: `ToolkitResponse` – contains the biometric authentication status.

4.2.20. Method: `resetPin`

Resets the ID Card PIN using the VG service. Involves biometric authentication. `prepareRequest` must be called first.

Syntax

```
resetPin(encodedPin, fingerData, sensorTimeout, appCallBack);
```

Parameter	Description
encodedPin	New PIN, encrypted and Base64-encoded per the Programmer's Reference.
fingerData	FingerData object from a previous <code>getFingerData</code> call.
sensorTimeout	Timeout in seconds for the fingerprint sensor.

Callback response: `ToolkitResponse` – contains the reset status.

4.2.21. Method: `resetPINWithoutAuthenticateBiometric`

Resets the ID Card PIN without biometric authentication. `prepareRequest` must be called first.

Syntax

```
resetPINWithoutAuthenticateBiometric(encodedPin, appCallBack);
```

Parameter	Description
encodedPin	New PIN, encrypted and Base64-encoded per the Programmer's Reference.

Callback response: `ToolkitResponse` – contains the reset status.

4.2.22. Method: `unlockPin`

Unlocks the ID Card PIN after multiple failed attempts. Involves biometric authentication. `prepareRequest` must be called first.

Syntax

```
unlockPin(encodedPin, fingerData, sensorTimeout, appCallBack);
```

Parameter	Description
encodedPin	New PIN, encrypted and Base64-encoded per the Programmer's Reference.
fingerData	FingerData object from a previous <code>getFingerData</code> call.
sensorTimeout	Timeout in seconds for the fingerprint sensor.

Callback response: `ToolkitResponse` – contains the unblock status.

4.2.23. Method: `readFamilyBookData`

Reads family book data from the card. Requires PIN authentication. `prepareRequest` must be called first.

Syntax

```
readFamilyBookData(encodedPin, appCallBack);
```

Parameter	Description
encodedPin	ID Card PKI PIN, encrypted and Base64-encoded per the Programmer's Reference.

Callback response: `CardFamilyBookData` – Contains `headOfFamily`, `Wives[]`, and `children[]`.

4.2.24. Method: `setNfcAuthenticationParameters (Card Attributes)`

Sets NFC authentication parameters using card attributes printed on the card surface.

Syntax

```
setNfcAuthenticationParameters(cardNumber, dateOfBirth, expiryDate, appCallBack);
```

Parameter	Description
cardNumber	Card number of the Emirates ID Card holder.
dateOfBirth	Date of birth in YYMMDD format. Example: 12th April 1956 = 560412.
expiryDate	Card expiry date in YYMMDD format. Example: 16th November 2027 = 271116.

Callback response: `'success'` on success.

4.2.25. Method: `setNfcAuthenticationParameters (MRZ)`

Sets NFC authentication parameters using the MRZ string from the back of the ID card.

Syntax

```
setNfcAuthenticationParameters(mrzData, appCallBack);
```

Parameter	Description
mrzData	Scanned MRZ data from the back of the ID Card.

Callback response: 'success' on success.

4.3. ParseEFData

4.3.1. Method: parseEFData

Parses Elementary File data into a JSON string. This is a standalone function, not part of the CardReader Class.

Syntax

```
parseEFData(ef_data, appCallBack);
```

Parameter	Description
ef_data	Base64-encoded Elementary File data to parse (obtained from readPublicDataEF).

Callback response: String – parsed EF data as a JSON string.

5. Data Model Classes

5.1. Class: ToolkitException

Contains exception details from the Toolkit SDK.

Field	Description
code	Error code.
message	Error message.
exceptionType	Type of error: TOOLKIT_ERROR or CARD_PIN_ERROR.
attemptsLeft	Remaining PIN attempts before the card is blocked. Only meaningful when exceptionType is CARD_PIN_ERROR.

5.2. Class: ToolkitExceptionWithResponse

Extends ToolkitException with a response field. Contains exception details along with the Toolkit service response.

Field	Description
code	Error code.
message	Error message.
exceptionType	Type of error: TOOLKIT_ERROR or CARD_PIN_ERROR.
attemptsLeft	Remaining PIN attempts.
toolkit_response	Response received from the Toolkit Service.

5.3. Class: ToolkitResponse

Contains the response from a Toolkit service operation.

Field	Description
status	Status of the operation (success/fail).
response	Response received from the Toolkit Service.
cardNumber	Emirates ID Card number.
cardSerialNumber	Emirates ID Card serial number.
idNumber	Emirates ID number.
requestId	The request ID provided by the application.
service	Name of the requested service.

Field	Description
timeStamp	Timestamp when the response was signed by VG.
xmlString	Digitally signed Toolkit Response XML.

5.4. Class: RegisterDeviceResponse

Extends ToolkitResponse. Contains the response for registerDevice.

Field	Description
deviceRegistrationID	Device registration identifier from VG.

5.5. Class: CardCertificates

Extends ToolkitResponse. Contains PKI certificates from the card.

Field	Description
authenticationCertificate	Authentication certificate (Base64-encoded string).
signingCertificate	Signing certificate (Base64-encoded string).

5.6. Class: FingerData

Contains finger details for biometric operations.

Field	Description
fingerId	Finger index reference identifier.
fingerIndex	Finger index value: NO_MEANING, RIGHT_THUMB, RIGHT_INDEX, RIGHT_MIDDLE, RIGHT_RING, RIGHT_LITTLE, LEFT_THUMB, LEFT_INDEX, LEFT_MIDDLE, LEFT_RING, LEFT_LITTLE.

5.7. Class: SignatureResponse

Extends ToolkitResponse. Contains the result of signing operations.

Field	Description
signature	Base64-encoded digital signature.

5.8. Class: CardPublicData

Extends ToolkitResponse. Contains public data read from the card.

Field	Description
cardNumber	Emirates ID card number.
cardHolderPhoto	Card holder photograph (Base64-encoded JPEG).

Field	Description
holderSignatureImage	Handwritten signature image (Base64-encoded).
modifiablePublicData	ModifiablePublicData object. Refer to the Programmer's Reference for attributes.
nonModifiablePublicData	NonModifiablePublicData object. Refer to the Programmer's Reference for attributes.
homeAddress	HomeAddress object. Refer to the Programmer's Reference for attributes.
workAddress	WorkAddress object. Refer to the Programmer's Reference for attributes.

Note: The `holderSignatureImage` field returned by the Validation Gateway is in TIFF format (CCITT compressed). Web browsers cannot render TIFF images natively. Applications requiring display of the holder signature image should convert the TIFF data to a browser-compatible format (PNG or JPEG) before rendering. The raw TIFF data is still included in the XML response and accessible to the application.

5.9. Class: CardFamilyBookData

Extends `ToolkitResponse`. Contains family book data from the card.

Field	Description
headOfFamily	HeadOfFamily object. Refer to the Programmer's Reference for attributes.
Wives	Array of Wife objects.
children	Array of Child objects.

6. Advanced Digital Signature API Reference

6.1. Class: CardReader (Digital Signature Methods)

6.1.1. Method: padesign

Digitally signs a PDF document in conformance with PAdES per ETSI EN 319 142. `prepareRequest` must be called first.

Syntax

```
padesign(signingContext, pdfFilePath, signedPdfFilePath, appCallBack);
```

Parameter	Description
signingContext	Signature properties including PIN and level. Refer to the Programmer's Reference for details.
pdfFilePath	Path of the input PDF document.
signedPdfFilePath	Path to store the signed PDF.

Callback response: `ToolkitResponse` – contains the signing status. The signed file is written to `signedPdfFilePath`.

6.1.2. Method: padesVerify

Verifies a PAdES-signed PDF document.

Syntax

```
padesVerify(verificationContext, signedPdfFilePath, appCallBack);
```

Parameter	Description
verificationContext	Verification context properties. Refer to the Programmer's Reference for details.
signedPdfFilePath	Path of the signed PDF to verify.

Callback response: `String` – verification report in XML format.

6.1.3. Method: xadesSign

Digitally signs an XML document in conformance with XAdES per ETSI EN 319 132. `prepareRequest` must be called first.

Syntax

```
xadesSign(signingContext, xmlFilePath, signedXmlFilePath, appCallBack);
```

Parameter	Description
signingContext	Signature properties including level, packaging mode, and PIN.
xmlFilePath	Path of the input XML document.
signedXmlFilePath	Path to store the signed document. Pass <code>null</code> for detached mode.

Callback response: `ToolkitResponse` – in detached mode, `sign_data` contains the Base64-encoded signature. In enveloped/enveloping mode, the signature is embedded in the output file.

Signing Mode	signedXmlFilePath	sign_data
Detached	<code>null</code>	Base64-encoded signature
Enveloped / Enveloping	Path to signed document	Not set

6.1.4. Method: `xadesVerify`

Verifies an XAdES-signed XML document.

Syntax

```
xadesVerify(verificationContext, signedXmlFilePath, signature, appCallBack);
```

Parameter	Description
verificationContext	Verification context properties.
signedXmlFilePath	Path of the XML document to verify.
signature	Base64-encoded detached signature (for detached mode). Pass <code>null</code> for enveloped/enveloping.

Callback response: `String` – verification report in XML format.

6.1.5. Method: `codesSign`

Digitally signs data in conformance with CADES per ETSI EN 319 122. `prepareRequest` must be called first.

Note: CADES signing works only in detached mode.

Syntax

```
codesSign(signingContext, inputFilePath, appCallBack);
```

Parameter	Description
signingContext	Signature properties. Packaging mode must be Detached.

Parameter	Description
inputFilePath	Path of the input file to sign.

Callback response: ToolkitResponse – sign_data contains the Base64-encoded CAdES signature.

6.1.6. Method: cadesVerify

Verifies a CAdES signature.

Syntax

```
cadesVerify(verificationContext, inputFilePath, signature, appCallBack);
```

Parameter	Description
verificationContext	Verification context properties.
inputFilePath	Path of the original document.
signature	Base64-encoded CAdES signature to verify.

Callback response: String – verification report in XML format.